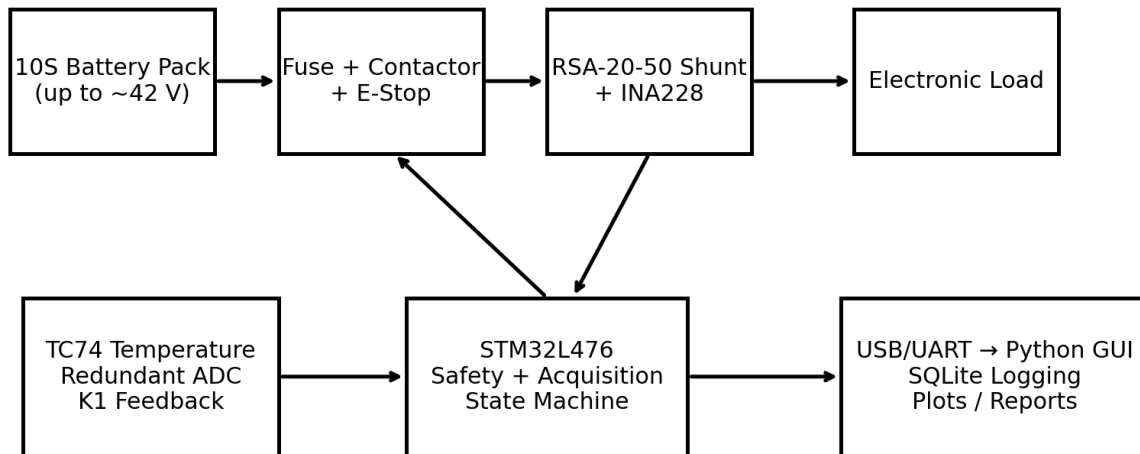


OSBAMS: Open-Source Battery Assessment and Management System

Joe Le — Computer Engineering

OSBAMS High-Level Architecture



Project Description

OSBAMS is a low-cost, open-source platform for figuring out whether a used lithium-ion pack is actually still good. It's built around a custom control PCB and an STM32L476 Nucleo controller, with precision voltage/current measurement, pack-temperature sensing, a high-current shunt, a hardware contactor/E-stop chain, an external electronic load, and Python/SQLite software running on the host side. The point is to run controlled discharge tests, log voltage, current, power, and temperature over time, and come away with an actual capacity and health estimate — something closer to real data than the usual “does the pack still work?” guess.

Significance

A reused battery pack can still have plenty of life left in it, but nobody really knows its history — how it was charged, how hard it was used, whether it was ever abused. Commercial cyclers and lab-grade DAQ systems can answer that question, but they're expensive and mostly closed boxes. OSBAMS is meant to be the opposite: an open, student-built platform where the hardware, firmware, safety logic, calibration process, and data analysis are all visible and modifiable. Along the way it touches embedded systems, PCB design, instrumentation, safety

engineering, serial protocols, Python, databases, and experimental validation — basically the full stack of a real measurement project.

Approach

The approach centers on an STM32L476, with hardware and firmware designed together rather than bolted on afterward. Battery current runs through a fuse, a contactor, an external RSA-20-50 shunt, and the electronic load. An INA228 handles the precision voltage/current/power measurement, a TC74 covers temperature, and a separate ADC divider acts as an independent voltage cross-check in case the INA228 is wrong or dead. On the firmware side: timed acquisition, sensor validation, fault handling, a watchdog, E-stop/ARM logic, and UART telemetry out to the host. A Python desktop app reads those OSBAMS frames, writes them into SQLite, plots the test in progress, scores the pack's condition, and exports a report. Nothing gets near a real battery until it's been tested logic-only over USB, then at low voltage on the bench, then finally staged up to full battery tests.

Innovation

What sets OSBAMS apart isn't raw precision — it's not trying to beat an industrial cyclor at its own game. It's the combination of open, inspectable safety hardware, redundant sensing, and a data path that runs unbroken from the embedded side to the desktop. Transparency, low cost, and modularity matter more here than squeezing out another decimal point of accuracy, especially for a tool aimed at assessing reused packs. The architecture keeps real-time safety decisions on the STM32, where they belong, and leaves visualization, storage, and analysis to the host computer.

Estimated Budget and Funding Source

Prototype hardware comes to roughly \$333, though several of the major parts were already owned or purchased before this proposal, so the actual new spending is lower. Funding so far has been out of pocket. Reusing the Nucleo, the electronic load, existing battery hardware, and ordering the PCB as a five-board batch all helped keep the incremental cost down.

Anticipated Pitfalls and Mitigation Plan

The real risks here are wiring or polarity mistakes, I²C sensor faults, calibration drift, a contactor or E-stop that doesn't do what it's supposed to, electrical noise, bugs in the state machine, and just plain unsafe testing. To deal with that: current limiting and fusing, an E-stop that's hardwired rather than software-mediated, a contactor that defaults off, a watchdog with latched fault states, redundant voltage sensing, proper Kelvin shunt wiring, staged low-voltage testing before anything else, independent multimeter checks, calibration before full power, and — most importantly — never trusting unvalidated software as the only thing standing between the system and a hazard.

Team

Right now it's just me — Joe Le, Computer Engineering — handling system architecture, PCB integration, embedded firmware, desktop software, and validation. I'm requesting Dr. George Anwar as faculty advisor. If this ever grows into a group project, the natural roles to add would be someone on embedded/firmware validation, someone on power/safety hardware, and someone on the desktop/data-analysis side.

Extra Notes

Where things stand right now: the custom PCB and most of the physical wiring are assembled, the STM32 firmware builds and flashes cleanly, INA228 communication is up, and UART boot/status telemetry is working. What's left is bringing up the TC74 temperature sensor and finishing verification on power, E-stop, contactor feedback, the redundant ADC, calibration, and the staged test procedure — all before any battery actually gets connected.